

WEEK -3 DIVIDE AND CONQUER

1-NUMBER OF ZEROES IN A GIVEN ARRAY

Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

```
1 #include <stdio.h>
2
3 int countZeroes(int arr[], int low, int high, int n)
4 {
5     if (low > high)
6         return 0;
7     int mid = (low + high) / 2;
8     if (arr[mid] == 0 && (mid == 0 || arr[mid - 1] == 1))
9     {
10         return n - mid;
11     }
12     if (arr[mid] == 1)
13     {
14         return countZeroes(arr, mid + 1, high, n);
15     }
16     return countZeroes(arr, low, mid - 1, n);
17 }
18
19 int main()
20 {
21     int m;
22     scanf("%d", &m);
23
24     int arr[m];
25
26     for (int i = 0; i < m; i++)
27     {
28         scanf("%d", &arr[i]);
29     }
30
31     printf("%d", countZeroes(arr, 0, m - 1, m));
32
33     return 0;
34 }
```

	Input	Expected	Got	
	5 1 1 1 0 0	2	2	
	10 1 1 1	0	0	

	Input	Expected	Got	
	1 1 1 1 1 1 1 1			
	8 0 0 0 0 0 0 0 0 0	8	8	
	17 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0	2	2	

Passed all tests!

2-MAJORITY ELEMENT

Given an array **nums** of size **n**, return *the majority element*.

The majority element is the element that appears more than $\lfloor n / 2 \rfloor$ times. You may assume that the majority element always exists in the array.

Example 1:

Input: **nums** = [3,2,3]

Output: 3

Example 2:

Input: **nums** = [2,2,1,1,1,2,2]

Output: 2

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 104`
- `-231 <= nums[i] <= 231 - 1`

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

```

1  #include <stdio.h>
2  int count(int arr[], int left, int right, int value)
3  {
4      int i, c = 0;
5      for (i = left; i <= right; i++)
6      {
7          if (arr[i] == value)
8              c++;
9      }
10     return c;
11 }
12 int majorityElement(int arr[], int left, int right)
13 {
14     if (left == right)
15         return arr[left];
16     int mid = left + (right - left) / 2;
17     int leftMajority = majorityElement(arr, left, mid);
18     int rightMajority = majorityElement(arr, mid + 1, right);
19     if (leftMajority == rightMajority)
20         return leftMajority;
21     int leftCount = count(arr, left, right, leftMajority);
22     int rightCount = count(arr, left, right, rightMajority);
23
24     if (leftCount > rightCount)
25         return leftMajority;
26     else
27         return rightMajority;
28 }
29 int main()
30 {
31     int n;
32     scanf("%d", &n);
33     int nums[n];
34     for (int i = 0; i < n; i++)
35     {
36         scanf("%d", &nums[i]);
37     }
38     printf("%d", majorityElement(nums, 0, n - 1));
39     return 0;
40 }

```

Input	Expected	Got	
3 3 2 3	3	3	

Passed all tests!

Correct

Marks for this submission: 1.00/1.00.

3-FINDING FLOOR VALUE

Problem Statement:

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

```
1  #include <stdio.h>
2  int findFloor(int arr[], int low, int high, int x)
3  {
4      if (low > high)
5          return -1;
6      int mid = low + (high - low) / 2;
7      if (arr[mid] == x)
8          return arr[mid];
9      if (arr[mid] < x)
10     {
11         int result = findFloor(arr, mid + 1, high, x);
12         if (result == -1)
13             return arr[mid];
14         return result;
15     }
16     return findFloor(arr, low, mid - 1, x);
17 }
18 int main()
19 {
20     int n, x;
21     scanf("%d", &n);
22     int arr[n];
23     for (int i = 0; i < n; i++)
24     {
25         scanf("%d", &arr[i]);
26     }
27     scanf("%d", &x);
28     int result = findFloor(arr, 0, n - 1, x);
29     printf("%d", result);
30     return 0;
31 }
```

Input	Expected	Got
6 1 2 8 10	2	2

	Input	Expected	Got	
	12 19 5			
	5 10 22 85 108 129 100	85	85	
	7 3 5 7 9 11 13 15 10	9	9	

Passed all tests!

Correct

4-TWO ELEMENTS SUM TO X

Problem Statement:

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as “No”.

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value “x”)

Answer:(penalty regime: 0 %)

```

1 #include <stdio.h>
2 int findPair(int arr[], int left, int right, int x, int *a, int *b)
3 {
4     if (left >= right)
5         return 0;
6     int sum = arr[left] + arr[right];
7     if (sum == x)
8     {
9         *a = arr[left];
10        *b = arr[right];
11        return 1;
12    }
13    if (sum < x)
14        return findPair(arr, left + 1, right, x, a, b);
15    return findPair(arr, left, right - 1, x, a, b);
16 }
17 int main()
18 {
19     int n, x;
20     scanf("%d", &n);
21     int arr[n];
22     for (int i = 0; i < n; i++)
23     {
24         scanf("%d", &arr[i]);
25     }
26     scanf("%d", &x);
27     int a, b;
28     if (findPair(arr, 0, n - 1, x, &a, &b))
29     {
30         printf("%d\n", a);
31         printf("%d\n", b);
32     }
33     else
34     {
35         printf("No");
36     }
37     return 0;
38 }

```

	Input	Expected	Got	
✓	4	4	4	✓
	2	10	10	
	4			
	8			
	10			
	14			
✓	5	No	No	✓
	2			
	4			
	6			
	8			
	10			
	100			

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

5-IMPLEMENTATION OF QUICK SORT

Write a Program to Implement the Quick Sort Algorithm

Input Format:

The first line contains the no of elements in the list-n

The next n lines contain the elements.

Output:

Sorted list of elements

For example:

Input	Result
5 67 34 12 98 78	12 34 67 78 98

```

1 #include <stdio.h>
2 void swap(int *a, int *b)
3 {
4     int temp = *a;
5     *a = *b;
6     *b = temp;
7 }
8 int partition(int arr[], int low, int high)
9 {
10     int pivot = arr[high];
11     int i = low - 1;
12     for (int j = low; j < high; j++)
13     {
14         if (arr[j] <= pivot)
15         {
16             i++;
17             swap(&arr[i], &arr[j]);
18         }
19     }
20     swap(&arr[i + 1], &arr[high]);
21     return i + 1;
22 }
23 void quickSort(int arr[], int low, int high)
24 {
25     if (low < high)
26     {
27         int pi = partition(arr, low, high);
28         quickSort(arr, low, pi - 1);
29         quickSort(arr, pi + 1, high);
30     }
31 }
32 int main()
33 {
34     int n;
35     scanf("%d", &n);
36     int arr[n];
37     for (int i = 0; i < n; i++)
38     {
39         scanf("%d", &arr[i]);
40     }
41     quickSort(arr, 0, n - 1);
42     for (int i = 0; i < n; i++)
43     {
44         printf("%d", arr[i]);
45         if (i < n - 1)
46             printf(" ");
47     }
48     return 0;
49 }

```

	Input	Expected	Got	
✓	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	✓
✓	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✓
✓	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✓

Passed all tests! ✓